ASSIGNMENT 1 REPORT

Design, Implement and Validate an Object-Oriented Model for a Campus Smart-Parking System

CO1: Java fundamentals and object-oriented programming

Name	G VENKATA GANESH
Register Number	192425381
Course	CSA0910 – Java Programming
GitHub repository	https://github.com/Ganesh401697/PROGRAMMING-IN-JAVA/tree/main/CO1%20ASSIGNMENT

TASK 1 – Understand the Problem and Identify the Objects (4 marks)

1.1 Inputs and outputs

Operation	Inputs	Outputs
Entry	Registration number, vehicle type, pass type, entry time	Allocated {floor, slot}, parking ticket with unique number, entry time
Exit	Ticket number or registration number, exit time, kWh used (EVs)	Released slot, duration, parking fee, charging fee
Reports	Day / floor selection	Occupancy per floor, total collection

1.2 Nouns → candidate classes, verbs → methods

Candidate class	Responsibility	Why it is needed
Vehicle (abstract)	Holds registration number; knows its hourly rate; calculates fee	Cars, bikes, EVs share data but differ in pricing – one abstraction avoids duplicate code
TwoWheeler, Car, ElectricCar	Supply the type-specific rate / charging behaviour	Pricing differs by vehicle type (Task 7)
ParkingSlot / slot array	Hold at most one vehicle	Enforces “one slot, one vehicle”
ParkingFacility	Owns floors and slots; allocates and releases slots	Single place that knows the layout (jagged array)
ParkingTicket	Ticket number, vehicle, slot, entry time, pass type	Links a vehicle to a slot in time; proof for billing
Pass / user category	Student, staff, visitor – free or paid	Passes change the fee rule, not the vehicle
Chargeable (interface)	chargingFee(kWh)	Only EVs have this capability

Verbs → methods: allocate(), release(), issueTicket(), calculateFee(), chargingFee(), isFull(), findTicket().

1.3 Business rules

- A slot cannot hold two vehicles; a vehicle cannot occupy two slots.
- A ticket number is unique and never reused (static counter, only incremented).
- A vehicle with an active ticket cannot enter again (duplicate registration rejected).
- Exit requires an active ticket; releasing a free or non-existent slot is rejected.
- Duration cannot be negative; every started hour is billed in full.

- Valid monthly-pass holders pay no parking fee; visitors pay the full fee; EV charging is billed separately.

1.4 Assumptions

- Facility layout is fixed at start-up: floors can have different slot counts (jagged array).
- Slot search is first-free (floor 0, slot 0 upward). Time is measured in whole minutes.
- Rates (Rs/hour): two-wheeler 10, car 30, EV 30 + Rs 12 per kWh. Pass validity is checked by the caller.

Alternative Concept and Complexity Analysis – parallel Strings vs objects

```
String[] regNo = new String[n];   String[] type = new String[n];   long[] entryTime = new long[n];

int find(String r){ for(int i=0;i<count;i++) if(regNo[i].equals(r)) return i; return -1; }
```

Criterion	Parallel arrays	Vehicle objects
Find by regNo	O(n) linear scan	O(n) scan of Vehicle[]; O(1) average if HashMap<String,Vehicle>
Add new attribute	New array and edits everywhere arrays are shifted/copied (easy to desynchronise)	Add one field in one class
Space per vehicle	k attributes → k slots in k arrays (≈ 4–8 B each reference + the data)	One object: header ≈ 12–16 B + k fields; same O(k) per vehicle
Type safety / behaviour	Everything is a String; no methods	Fee logic travels with the object

Conclusion: asymptotically search is O(n) in both, and space is O(n·k) in both; the object design wins on maintainability and allows the O(1) HashMap index.

TASK 2 – Choose the Data Representation (3 marks)

Field	Type	Reason
Slot number / floor	int	Small whole numbers used as array indexes
Vehicle number	String	Alphanumeric (TN01AB1111)
Entry time	int minutes since midnight (or LocalTime)	Simple arithmetic; avoids floating point
Duration	int minutes	Whole minutes; non-negative
Fee	double (or BigDecimal in real billing)	Rates can have paise; document the rounding policy
Ticket number	int (static counter)	Unique, increasing

Floors and slots

A jagged array String[][] slots is used: slots[f] = new String[slotsPerFloor[f]]. It supports 1×20, 3×50, 5×100 and the Task 12 requirement of a different slot count on every floor, and wastes no memory (a rectangular array would need the largest floor size for every floor).

Type conversion and the casting risk

```
int minutes = 130;
double wrong = (minutes / 60) * 30;        // 130/60 = 2 (integer division) -> 60.0   WRONG (should be 90)
double wrong2 = (double)(minutes / 60) * 30; // cast applied too late, still 60.0
int hours = (minutes + 59) / 60;           // ceiling with integers only = 3
double fee = hours * 30.0;                 // 90.0   CORRECT
```

Test evidence: 61 minutes at Rs 30/h. The integer-division trap gives (61/60)*30 = 30.0, but the correct bill is 2 started hours = Rs 60. Our test table (Task 11) shows 61 min → 60.0 PASS and records the wrong value 30.0 for the trap. The design avoids the error by computing the ceiling with (minutes + 59) / 60 in one place (Vehicle.calculateFee) and by never dividing before multiplying by the rate.

Input

Scanner(System.in) is used for interactive entry (type, registration number, minutes) because the values are not known when the program starts. Command-line arguments are suitable only for fixed configuration such as the slots-per-floor array. Numeric input is read with Integer.parseInt inside try/catch to reject bad data.

Alternative Concept and Complexity Analysis – 1-D array and ArrayList

```
// 1-D: index = floor * slotsPerFloor + slot   (only valid when every floor has the SAME size)
String[] all = new String[F * S];
// uneven floors -> need offset[] : index = offset[floor] + slot,  offset[f+1] = offset[f] + size[f]

ArrayList<String[]> floors = new ArrayList<>();  floors.add(new String[20]);  floors.add(new String[35]);
```

	Jagged String[][]	**1-D with formula / offset[]**	**ArrayList<String[]>**
Different slots per floor	Natural	Formula breaks; needs offset[] (extra O(F))	Natural
Locate slot (f,s)	O(1)	O(1) (arithmetic + offset lookup)	O(1) (get(f) then [s])
Space	F row headers + ΣS refs	ΣS refs + F offsets	Row arrays + list backing array (extra ≈ F, up to 1.5F capacity)
Add a floor at run time	Copy outer array O(F)	Copy whole array O(ΣS)	Amortised O(1)

TASK 3 – Design the Control Logic (4 marks)

3.1 Slot search

Nested for loops walk floor 0…F-1 and, inside each floor, slot 0…S-1. The first null slot ends the search immediately with return {floor, slot} (return exits both loops – no flag or extra break needed). If both loops finish, every slot is occupied and the method returns null (the caller rejects the entry: "Facility full"). continue is used to skip slots that are not suitable (for example a charging slot when a non-EV arrives).

3.2 Fee slabs

A started hour is billed in full. Example slab policy: first hour = base rate; each additional hour = base rate; daily maximum = 8 × rate. A switch on the vehicle type is avoided in later tasks (polymorphism), but the slab itself is expressed with if–else:

```
hours = (minutes + 59) / 60
fee   = hours * rate
if (fee > dailyMax) fee = dailyMax
```

3.3 Pseudocode

```
ALGORITHM AllocateSlot(slots[][], vehicleNo)
INPUT  : slots – jagged array of slot occupants; vehicleNo – String
OUTPUT : pair (floor, slot) or NULL
BEGIN
 1. FOR f ← 0 TO LENGTH(slots) − 1 DO
 2.    FOR s ← 0 TO LENGTH(slots[f]) − 1 DO
 3.       IF slots[f][s] = NULL THEN
 4.          slots[f][s] ← vehicleNo
 5.          RETURN (f, s)
 6.       END IF
 7.    END FOR
 8. END FOR
 9. RETURN NULL          // facility full
END

ALGORITHM ComputeFee(minutes, ratePerHour, dailyMax)
INPUT  : minutes ≥ 0 (integer), ratePerHour > 0, dailyMax > 0
OUTPUT : fee (real)
BEGIN
 1. IF minutes < 0 THEN RAISE InvalidDuration
 2. hours ← (minutes + 59) DIV 60        // integer ceiling
 3. fee   ← hours × ratePerHour
 4. IF fee > dailyMax THEN fee ← dailyMax
 5. RETURN fee
END
```

(The brief refers to the "SIMATS Engineering – Standard for Algorithmic Pseudocode". The layout above – named algorithm, INPUT/OUTPUT, numbered steps, END keyword – is the usual form; adjust keywords to your copy of the standard.)

3.4 Complete the Code – ParkingFacility.java (TODOs filled)

```
public class ParkingFacility {
   // slots[floor][slot] holds a vehicle number, or null if the slot is free
   private final String[][] slots;


   public ParkingFacility(int[] slotsPerFloor) {
```

```
        slots = new String[slotsPerFloor.length][];
        for (int f = 0; f < slotsPerFloor.length; f++) {
            slots[f] = new String[slotsPerFloor[f]];
        }
    }

    /** Returns {floor, slot} of the first free slot, or null if it is full. */
    public int[] allocate(String vehicleNo) {
        for (int f = 0; f < slots.length; f++) {
            for (int s = 0; s < slots[f].length; s++) {
                if (slots[f][s] == null) {
                    slots[f][s] = vehicleNo;
                    return new int[]{f, s};
                }
            }
        }
        return null;
    }

    /** Frees a slot. Returns false if the position is invalid or already free. */
    public boolean release(int floor, int slot) {
        if (floor < 0 || floor >= slots.length) return false;
        if (slot < 0 || slot >= slots[floor].length) return false;
        if (slots[floor][slot] == null) return false;
        slots[floor][slot] = null;
        return true;
    }

    public static void main(String[] args) {
        // floor 0 has 2 slots, floor 1 has 1 slot
        ParkingFacility p = new ParkingFacility(new int[]{2, 1});
        System.out.println(java.util.Arrays.toString(p.allocate("TN01AB1111")));
        System.out.println(java.util.Arrays.toString(p.allocate("TN02CD2222")));
        System.out.println(java.util.Arrays.toString(p.allocate("TN03EF3333")));
        System.out.println(java.util.Arrays.toString(p.allocate("TN04GH4444")));
        System.out.println(p.release(0, 1) + " " + p.release(0, 1) + " "
                + p.release(5, 0));
        System.out.println(java.util.Arrays.toString(p.allocate("TN04GH4444")));
    }
}
```

Output:

```
[0, 0]
[0, 1]
[1, 0]
null
true false false
[0, 1]
```

Output matches the expected output exactly. The fourth allocation returns null because all 3 slots are taken; release(0,1) succeeds once, fails the second time (already free) and release(5,0) fails (invalid floor), so the last allocation reuses slot [0, 1].

Alternative Concept and Complexity Analysis – free counts + stack of free slots

```
int[] freeCount = new int[F];        // free slots on each floor
Deque<Integer>[] freeStack = ...;    // free slot numbers per floor (initially all pushed)

allocate(v): for f in 0..F-1: if freeCount[f] > 0 { s = freeStack[f].pop(); freeCount[f]--; slots[f][s]=v; return (f,s); }  return null
release(f,s): validate; slots[f][s]=null; freeStack[f].push(s); freeCount[f]++
```

	Scan approach	Counts + stack	Note
allocate()	O(F·S) worst case	O(F) (O(1) with one global stack)	Skips full floors without looking inside
release()	O(1)	O(1)	Both validate in constant time
Extra space	O(1)	O(F) counters + O(F·S) stack entries	Speed bought with memory

TASK 4 – Design Classes, Objects and Constructors (4 marks)

```
class ParkingTicket {
   private static int counter = 1000;
   private final int ticketNo;
   private final String regNo;
   private final int floor, slot, entryMinute;

   ParkingTicket(String regNo, int floor, int slot, int entryMinute) {   // parameterised (master)
      if (regNo == null || regNo.isBlank()) throw new IllegalArgumentException("regNo");
      this.ticketNo = ++counter;
      this.regNo = regNo; this.floor = floor; this.slot = slot; this.entryMinute = entryMinute;
   }
   ParkingTicket(String regNo, int floor, int slot) {              // overloaded: entry time = now
      this(regNo, floor, slot, currentMinute());     // constructor chaining with this(...)
   }
   ParkingTicket(ParkingTicket t) {                                // copy constructor (duplicate ticket)
      this.ticketNo = t.ticketNo;                 // SAME number: it is a re-print, not a new ticket
      this.regNo = t.regNo; this.floor = t.floor; this.slot = t.slot; this.entryMinute = t.entryMinute;
   }
   private static int currentMinute() { var n = java.time.LocalTime.now(); return n.getHour()*60 + n.getMinute(); }
}
```

Class	Fields / methods / constructors
Vehicle (abstract)	private final regNo; getRegNo(); abstract ratePerHour(); calculateFee(int); equals/hashCode/toString. Protected constructor Vehicle(String).
ParkingFacility	private String[][] slots; constructor ParkingFacility(int[] slotsPerFloor); allocate(), release(), isFull(). No default constructor – a facility without a layout is meaningless.
ParkingTicket	As above – three constructors.

Default constructor: used only where a valid empty state exists (e.g. a report object). Parameterised: whenever mandatory data must exist at birth (ticket, vehicle). Overloaded: convenience variants. this(...) chaining keeps validation and ticket-number assignment in ONE constructor. A copy constructor is needed for re-issuing a lost ticket: it duplicates data but must not consume a new ticket number.

Memory

Reference variables (e.g. ParkingTicket t, Vehicle v) live on the stack frame of the method; the objects themselves live on the heap. Vehicle[] parked is a heap array of references; each element points to a heap object of the actual subclass. Static fields (counter) are held once per class, not per object. When a method returns, its stack references vanish; the heap object survives while another reference (e.g. in the facility's ticket map) still points to it.

Alternative Concept and Complexity Analysis – setters or static factories

```
ParkingTicket t = new ParkingTicket();  t.setRegNo("TN01"); t.setSlot(0,1);        // setters: object exists in an INVALID state in between

static ParkingTicket issue(String reg, int f, int s){ return new ParkingTicket(reg, f, s); }  // factory
```

	Chained constructors	Default + setters	Static factory
Guarantees valid object	Yes, at creation	No – caller may forget a setter	Yes (can also return cached/sub-types)

	Chained constructors	Default + setters	Static factory
Fields can be final	Yes	No (must be mutable)	Yes
Time for n tickets	O(n)	O(n·k) – k setter calls each	O(n)
Space	O(n)	O(n)	O(n) (plus optional cache)

TASK 5 – static Members and the Object Life Cycle (3 marks)

- static int counter in ParkingTicket – one counter for the whole class; each ticket must get the next number, so an instance variable (reset to 0 in every object) would create duplicate numbers.
- static int totalOccupancy (or a facility-wide count) – describes the facility as a whole, not one ticket.
- static methods: utility behaviour that needs no object, e.g. ParkingApp.hours(int minutes) and the factory make(type, reg). A static block can initialise a fee table once when the class loads.

```
static final Map<String,Double> RATE = new HashMap<>();
static { RATE.put("2W", 10.0); RATE.put("CAR", 30.0); RATE.put("EV", 30.0); }   // runs once at class load
```

Life cycle and object passing

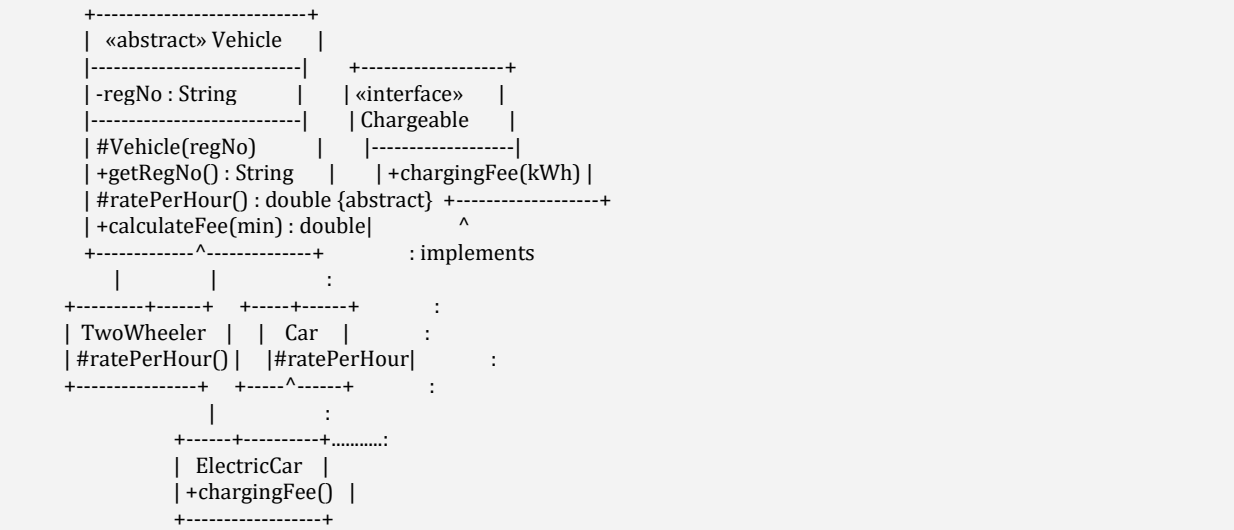
A ParkingTicket is created on entry and stored in the active-ticket map. On exit it is removed (active.remove(regNo)); once the method finishes and no variable refers to it, it is unreachable and eligible for garbage collection (the JVM decides when it is actually collected). Java passes everything by value: for objects the value is a copy of the reference, so a method can modify the object's state (e.g. slots[f][s]=null) but cannot make the caller's variable point somewhere else. Methods return references (Ticket enter(...)) or arrays (int[] allocate(...)).

Alternative Concept and Complexity Analysis – instance counter

If counter were an instance variable of ParkingFacility, the north-gate and south-gate facilities would each start at 1001 and both issue "ticket 1001", violating the rule that ticket numbers are never reused. Solutions: (a) keep a static counter shared by all facilities (simple, one number series; needs synchronisation or AtomicInteger if multi-threaded), or (b) keep an instance counter and prefix the facility code (N-1001, S-1001) so numbers are unique per facility.

Space for k facilities, n tickets	**Static counter**	**Instance counter**
Counter storage	O(1)	O(k)
Tickets	O(n)	O(n)
Uniqueness	Global	Per facility (needs prefix)

TASK 6 – Design the Inheritance Hierarchy (4 marks)



Type of inheritance: hierarchical (TwoWheeler and Car both extend Vehicle) combined with multilevel (ElectricCar → Car → Vehicle). A class may extend only one class, so the EV capability is added by implementing the Chargeable interface. Users (Student, Staff, Visitor) are NOT modelled as subclasses of Vehicle: a user is not a vehicle, and the same person can bring different vehicles. A pass type (enum) is attached to the ticket instead.

Inherited vs added

Class	Inherited	Added / overridden
TwoWheeler	regNo, getRegNo(), calculateFee()	ratePerHour() = 10
Car	same	ratePerHour() = 30
ElectricCar	everything from Car	chargingFee(kWh) = 12 × kWh

super is used (i) in constructors – super(regNo) must be the first statement so the private regNo of Vehicle is initialised; (ii) for methods – super.calculateFee(m) lets a subclass reuse the parent algorithm; (iii) for fields – super.field when a subclass hides one (avoided here because regNo is private).

Constructor execution trace: new ElectricCar("TN03C3")

Step	Constructor	Action
1	ElectricCar(regNo) starts	First statement is the implicit/explicit super(regNo) → control goes up
2	Car(regNo) starts	Calls super(regNo) → goes up
3	Vehicle(regNo) (Object() runs first)	Object() runs, then regNo is assigned in Vehicle
4	Vehicle completes	Returns to Car
5	Car completes	Returns to ElectricCar
6	ElectricCar body runs	Object fully built. Order of execution: Object → Vehicle → Car → ElectricCar

Alternative Concept and Complexity Analysis – composition

```
class VehicleType { final String name; final double rate; }
class Vehicle { final String regNo; final VehicleType type; double fee(int m){ return ((m+59)/60) * type.rate; } }
```

	Inheritance	**Composition**
Add a new vehicle type	New subclass (new code, recompile)	New VehicleType object / data row – no new class
Time to compute fee	O(1) – virtual call	O(1) – one extra field read
Space per Vehicle	regNo only (rate is in code)	regNo + 1 reference (4–8 B); VehicleType objects are shared, O(k) total
Type-specific behaviour (charging)	Natural via overriding / interfaces	Needs strategy objects

TASK 7 – Override and Specialise Behaviour (4 marks)

ratePerHour() is abstract in Vehicle and overridden in every concrete class because the price is exactly what differs between types; calculateFee() lives once in Vehicle and calls the overridden ratePerHour() (template-method style). Subclasses such as EBike and DeliveryVan (Task 12) override calculateFee() itself because their rule is not "hours × rate".

```
@Override protected double ratePerHour() { return 30; }
```

- @Override makes the compiler verify that the method really overrides – a typo (rateperHour) becomes a compile error instead of a silent new method.
- Rules: same name and parameter list; return type same (or covariant); access cannot be narrower (protected → protected/public); cannot throw broader checked exceptions; static, private and final methods cannot be overridden.
- final: final String regNo (set once in the constructor; a vehicle never changes its number); final method for a fee-audit method that must not be altered; final class for an immutable value class such as a Money type.

```
@Override public boolean equals(Object o){ return o instanceof Vehicle v && regNo.equals(v.regNo); }
@Override public int hashCode(){ return regNo.hashCode(); }      // equal objects MUST have equal hash codes
@Override public String toString(){ return getClass().getSimpleName() + "[" + regNo + "]"; }
```

Because equals compares only regNo, two Vehicle objects with the same registration are equal, and HashMap/HashSet treat them as the same key. hashCode is derived from the same field to keep the equals/hashCode contract.

Alternative Concept and Complexity Analysis – switch on a type field

```
double calculateFee(int minutes){
   int h = (minutes + 59) / 60;
   switch (type) { case TWO_WHEELER: return h*10; case CAR: return h*30; case EV: return h*30; /* ... */ default: throw new
IllegalStateException(); }
}
```

	Overriding	**switch on type**
Time per fee	O(1) (virtual dispatch)	O(1) for a compiled switch; O(k) if written as if–else chain
Classes changed to add a type	0 existing (1 new class)	1 existing class edited – and every other switch on type in the code base
Risk	Low	Forgotten case → wrong fee or exception

TASK 8 – Apply Polymorphism (4 marks)

Compile-time polymorphism (overloading)

```
public double calculateFee(int minutes)                 { ... }
public double calculateFee(LocalTime in, LocalTime out)       { return calculateFee((int) Duration.between(in,
out).toMinutes()); }
public double calculateFee(int minutes, int entryHour)        { ... }   // used in Task 12
```

The compiler chooses by the number and static types of the arguments: calculateFee(130) → (int); calculateFee(t1, t2) → (LocalTime, LocalTime). This is resolved at compile time.

Runtime polymorphism

Vehicle[] parked = { new TwoWheeler(..), new Car(..), new ElectricCar(..) } is upcasting: each element is stored in a Vehicle reference. The call v.calculateFee(130) is resolved at run time from the actual object. There is no if/else on the vehicle type anywhere in the loop.

Trace of one call

Statement	**v.calculateFee(130) in the loop, 3rd element**
Declared (static) type of v	Vehicle – the compiler only checks that Vehicle has calculateFee(int)
Actual (runtime) object type	ElectricCar
Method executed	Vehicle.calculateFee(int) (inherited; not overridden in Car/ElectricCar). Inside it, hours = (130+59)/60 = 3, then ratePerHour() is dispatched dynamically → Car.ratePerHour() = 30 → 3 × 30 = 90.0
Then	v instanceof Chargeable c is true → + c.chargingFee(10) = 120 → total 210.0

Complete the Code – FeeDemo.java

```
interface Chargeable {
   double chargingFee(int kWh);
}

abstract class Vehicle {
   private final String regNo;
   protected Vehicle(String regNo) { this.regNo = regNo; }
   public String getRegNo() { return regNo; }
   protected abstract double ratePerHour();

   /** Every started hour is charged in full. */
   public double calculateFee(int minutes) {
      int hours = (minutes + 59) / 60;         // ceiling using integer arithmetic only
      return hours * ratePerHour();
   }

   @Override public boolean equals(Object o) {
      return o instanceof Vehicle && regNo.equals(((Vehicle) o).regNo);
   }
   @Override public int hashCode() { return regNo.hashCode(); }
}

class TwoWheeler extends Vehicle {
   TwoWheeler(String regNo) { super(regNo); }
   @Override protected double ratePerHour() { return 10; }
}
```

```
class Car extends Vehicle {
   Car(String regNo) { super(regNo); }
   @Override protected double ratePerHour() { return 30; }
}

class ElectricCar extends Car implements Chargeable {
   ElectricCar(String regNo) { super(regNo); }
   @Override public double chargingFee(int kWh) { return kWh * 12; }
}

public class FeeDemo {
   public static void main(String[] args) {
      Vehicle[] parked = { new TwoWheeler("TN01A1"), new Car("TN02B2"),
            new ElectricCar("TN03C3") };
      for (Vehicle v : parked) {
         double fee = v.calculateFee(130); // dynamic method dispatch
         if (v instanceof Chargeable c) fee += c.chargingFee(10);
         System.out.println(v.getClass().getSimpleName() + " " + v.getRegNo()
               + " -> Rs " + fee);
      }
      System.out.println(new Car("TN02B2").equals(new ElectricCar("TN02B2")));
   }
}
```

Output:

```
TwoWheeler TN01A1 -> Rs 30.0
Car TN02B2 -> Rs 90.0
ElectricCar TN03C3 -> Rs 210.0
true
```

Calculation: 130 min → ceiling(130/60) = 3 hours. Two-wheeler 3×10 = 30; Car 3×30 = 90; EV 90 + 10 kWh × 12 = 210. Output matches the expected output.

Why does the last line print true?

equals() is inherited from Vehicle and compares only regNo, using instanceof so that a Car and its subclass ElectricCar are comparable. "TN02B2" equals "TN02B2", so the result is true. Is this correct for the system? Yes – a registration number identifies one physical vehicle, so two objects with the same number represent the same vehicle, and this is exactly what the duplicate-registration rule needs. (The result is also symmetric: new ElectricCar("TN02B2").equals(new Car("TN02B2")) is true.) The limitation is that a Car and a TwoWheeler with the same number would also be “equal”; that is acceptable because real registration numbers are unique across types.

Alternative Concept and Complexity Analysis – instanceof and casts

```
double fee;
if (v instanceof TwoWheeler) fee = ((TwoWheeler) v).calculateFee(130);
else if (v instanceof ElectricCar) fee = ((ElectricCar) v).calculateFee(130);
else if (v instanceof Car) fee = ((Car) v).calculateFee(130);   // order matters: subclass first!
```

	Dynamic dispatch	**instanceof chain**
Time per call	O(1)	O(k) – up to k checks
Code size	One call	k branches at every place the behaviour is needed

	Dynamic dispatch	**instanceof chain**
Adding a type	New class only	Edit every chain; a wrong order (Car before ElectricCar) silently gives wrong results

TASK 9 – Introduce Abstraction (4 marks)

- Vehicle is abstract: it is a concept, never a real object (nobody parks a "generic vehicle"), but it holds shared state (regNo), a constructor and the common algorithm calculateFee(). ratePerHour() is abstract: every concrete type must supply it.
- Chargeable is an interface: it is a capability that cuts across the hierarchy (an e-bike, an EV or a golf cart could all be charged) and has no state. Payable could be another interface for fee-paying users.
- Abstract class vs interface: use an abstract class for "is-a" with shared state and constructor code; use an interface for "can-do" capabilities. A class extends one class but may implement many interfaces – this is how multiple behaviours are obtained (class ElectricCar extends Car implements Chargeable, Payable).

```
interface Chargeable {
   double chargingFee(int kWh);
   default double chargingFee(int kWh, boolean peak) { return peak ? chargingFee(kWh) * 1.5 : chargingFee(kWh); } // default:
evolves interface without breaking implementers
   static boolean isValidKwh(int kWh) { return kWh >= 0; }                    // static: utility tied to the interface
}
```

Alternative Concept and Complexity Analysis – swap the roles

Change	Consequence
Vehicle → interface with default methods	No constructor; no instance state – regNo cannot be a private final field (only public static final constants), so getRegNo() must be abstract and every class re-declares the field; fields of implementors are the only state; protected members are not possible (interface members are public).
Chargeable → abstract class	ElectricCar already extends Car, and Java has no multiple class inheritance → ElectricCar could no longer extend Chargeable. The capability could not be shared across unrelated hierarchies.
Time per method call	O(1) in both; invokeinterface can be marginally slower than invokevirtual on cold paths but the JIT removes the difference – negligible.
Space per object	Unchanged – methods are stored per class, not per object; state fields only exist in the abstract-class design.

TASK 10 – Encapsulation through Packages and Access Modifiers (3 marks)

```
parking.model    -> Vehicle, TwoWheeler, Car, ElectricCar, EBike, DeliveryVan, ParkingTicket, PassType, Chargeable
parking.service  -> ParkingFacility, FeeCalculator
parking.app      -> ParkingApp (main), console I/O
```

Member	Modifier	Why
Vehicle.regNo	private final	Only the constructor may set it; reachable through getRegNo()
Vehicle.ratePerHour()	protected	Subclasses override it; outside code should call calculateFee(), not the raw rate
Vehicle.calculateFee()	public	The class's service to other packages
ParkingFacility.slots	private	Only allocate/release may change slot contents
Package-private helper (no modifier)	default	Used only by classes in parking.service
ParkingTicket constructor	public (or package-private if only the facility may issue tickets)	Controls who can create tickets

```
public void setDuration(int minutes){ if (minutes < 0) throw new IllegalArgumentException("duration must be >= 0");
this.minutes = minutes; }
public boolean release(int floor, int slot){ if (floor < 0 || floor >= slots.length) return false; if (slot < 0 || slot >=
slots[floor].length) return false; ... }
```

Wrapper classes and autoboxing: Map<String,Integer> and ArrayList<Integer> store Integer, so int ↔ Integer conversion happens automatically. Risks: (1) Integer a=1000, b=1000; a==b is false because == compares references (only −128..127 are cached) – always use equals(); (2) unboxing a null Integer throws NullPointerException; (3) autoboxing in loops allocates many objects.

Alternative Concept and Complexity Analysis – public fields

```
v.minutes = -50;          // nothing stops it
if (t.floor >= slots.length) ...   // every caller must repeat the check
```

With public fields, any code can set an invalid state (negative duration, slot 99, regNo null) and the object can no longer guarantee its own rules. Validation costs O(1) per update in both designs, but with encapsulation the check is written once; with public fields it must be repeated at each of the m places that modify the field (m grows with the program) and one forgotten check is a bug. Changing the representation later (e.g. minutes → Duration) would need edits in all m places instead of one.

TASK 11 – Implement, Test and Defend the Design (5 marks)

The complete system (ParkingApp.java, Appendix A) was executed. Test table with expected and actual results produced by the program:

Type	Test	Expected	Actual	Result
Normal	Entry of two-wheeler/car – ticket numbers increase	true	true	PASS
Normal	2W, 130 min	30.0	30.0	PASS
Normal	Car, 130 min	90.0	90.0	PASS
Normal	EV 130 min + 10 kWh	210.0	210.0	PASS
Boundary	Last slot used → next entry	IllegalStateException	IllegalStateException	PASS
Boundary	Zero minutes	0.0	0.0	PASS
Boundary	Exactly 60 min (slab boundary)	30.0	30.0	PASS
Boundary	61 min (1 min over)	60.0	60.0	PASS
Boundary	Slot reuse after exit	ok	ok	PASS
Invalid	Unknown vehicle type "BUS"	IllegalArgumentException	IllegalArgumentException	PASS
Invalid	Duplicate registration	IllegalStateException	IllegalStateException	PASS
Invalid	Exit with no ticket / invalid slot	NoSuchElementException	NoSuchElementException	PASS
Invalid	Negative duration	IllegalArgumentException	IllegalArgumentException	PASS

Hand-calculated daily collection

Veh.	Type / pass / time	Hand calculation	Fee (Rs)	Program
D1	Two-wheeler, 45 min	1 h × 10	10	10
D2	Car, 125 min	3 h × 30	90	90
D3	EV, 60 min, 5 kWh	1 h × 30 + 5 × 12	90	90
D4	E-bike, 30 min	flat	15	15
D5	Delivery van, 90 min, 10 AM	2 h × 60	120	120
D6	Car, monthly pass	free	0	0
	TOTAL	10+90+90+15+120+0	325	325.0 PASS

Output:

```
TOTAL PASS=20 FAIL=0    (all 20 automated checks passed, including the hand-calculated total 325.0)
```

Evaluation of the design

- Encapsulation: regNo private final, slots private; state changes only through validated methods.
- Inheritance: shared code in Vehicle; subtypes only supply their rate.
- Polymorphism: facility.exit() calls calculateFee() without testing the type.
- Abstraction: Vehicle abstract, Chargeable interface.

- Limitation: the facility still uses instanceof Chargeable and a switch in the make() factory; the pass logic is inside exit(); rates are hard-coded in classes (not configurable); a first-free search is O(F·S).

Alternative Concept and Complexity Analysis – ArrayList / HashMap for active tickets

```
Map<String,ParkingTicket> active = new HashMap<>();
ParkingTicket t = active.remove(regNo);   // O(1) average
// array / ArrayList: for (int i...) if (list.get(i).getRegNo().equals(regNo)) -> O(n), then remove(i) is another O(n) shift for ArrayList
```

Measured on this machine (find the LAST of n registrations; mean ns per lookup over 2000 repeats, plain Java, no warm-up – values are indicative only, JIT and GC cause noise):

n	Array scan	ArrayList.indexOf	HashMap.get
1,000	18,985 ns	21,851 ns	46 ns
10,000	121,851 ns	44,661 ns	3,686 ns (noisy first-touch)
100,000	497,460 ns	483,646 ns	35 ns

Structure	Find on exit	Space for n vehicles	Remark
Array	O(n)	n references (fixed size)	Need to know n in advance
ArrayList	O(n) (+O(n) shift on remove)	n refs, capacity up to ≈1.5 n	Grows automatically
HashMap	O(1) average	≈ n entries × (32–48 B) + table	Fastest; also rejects duplicate keys naturally

Recommendation: use HashMap<String,ParkingTicket> for active tickets. The array/list scan grows linearly (≈ 500 µs at 100,000 vehicles) while the map stays in tens of nanoseconds; the extra memory is a small price. The measurements confirm the Big-O analysis.

TASK 12 – Re-evaluate and Adapt the Design (8 marks)

A. Impact Analysis (2)

- Changed: two new vehicle types (e-bike with flat fee, delivery van with time-of-day pricing), a new one-day visitor pass, and an extended facility with 5 floors with different slot counts (e.g. 4, 6, 3, 8, 5).
- Affected: Vehicle (fee now depends on entry time → needs an entry-hour parameter), the fee code in the facility's exit(), pass handling in the ticket, the vehicle factory (make()). NOT affected: TwoWheeler, Car, ElectricCar, Chargeable.
- Different slots per floor: a rectangular String[3][50] or the 1-D formula floor × slotsPerFloor + slot would break. The jagged array chosen in Task 2 already supports it – only the constructor argument changes (new int[]{4,6,3,8,5}).
- Modified rather than extended: Vehicle (one new overload) and the facility/ticket (pass type); everything else is extended.

B. Evaluation of the Original Design (2)

- New vehicle types can be added by writing new subclasses only – confirmed: EBike and DeliveryVan are new classes extending Vehicle.
- Pricing is spread across classes (one rule per subclass) – good for "one rule per type", but the time-of-day rule needed a signature change in the base class; a central PricingRule would put all rules in one place at the cost of an extra abstraction.
- Existing tested classes stayed unchanged: TwoWheeler, Car and ElectricCar were not edited and their original tests (30.0 / 90.0 / 210.0) still pass.

C. Decision: MODIFY (retain the hierarchy, small targeted changes) (2)

```
// NEW classes only
static class EBike extends Vehicle {
   EBike(String r){ super(r); }
   protected double ratePerHour(){ return 0; }
   @Override public double calculateFee(int m, int h){ if (m<0) throw new IllegalArgumentException(); return 15; }   // flat fee
}
static class DeliveryVan extends Vehicle {
   DeliveryVan(String r){ super(r); }
   protected double ratePerHour(){ return 60; }
   @Override public double calculateFee(int m, int h){
      double rate = (h >= 9 && h < 17) ? 2*30 : 30;        // double the car rate 9 AM - 5 PM
      return ((m+59)/60) * rate; }
}
enum PassType { NONE, MONTHLY, ONE_DAY }                  // one-day visitor pass
// Facility.exit(): fee = (ticket.pass == PassType.NONE) ? vehicle.calculateFee(minutes, entryHour) : 0;
//              charging (EV) is still billed even with a pass.
```

Revised structure: Vehicle ← {TwoWheeler, Car ← ElectricCar, EBike, DeliveryVan}; Chargeable; PassType; Facility with String[5][] jagged array. What changed: +2 classes, +1 enum, +1 overload in Vehicle (calculateFee(minutes, entryHour)), pass check in Facility.exit(), +2 cases in the factory. If the pricing rules keep growing, the next step would be a PricingRule interface chosen per vehicle type – not justified yet by only two new rules.

D. Evidence-Based Defence (2)

#	Test (all in ParkingApp.java)	Expected	Actual
1	E-bike 300 min (flat)	15.0	15.0 PASS
2	Van 130 min, entry 10 AM (3 h × 2 × 30)	180.0	180.0 PASS
3	Van 130 min, entry 8 PM (3 h × 30)	90.0	90.0 PASS
4	Visitor one-day pass, car, 600 min	0.0	0.0 PASS
5	Monthly pass EV 10 kWh (charging still billed)	120.0	120.0 PASS
6	Facility {4,6,3,8,5} used for all of the above	works	works PASS

- Classes changed: 2 existing (Vehicle: one overload added; Facility: pass handling + factory case). 3 existing classes (TwoWheeler, Car, ElectricCar) and the interface were not touched; 2 classes and 1 enum were added.
- Future vehicle types: write one subclass + one factory line; no other class changes (e.g. “bus” in 10 minutes).
- Trade-offs: a PricingRule/strategy design would centralise all pricing and could avoid even the two edits, but costs more classes today; keeping the hierarchy was cheaper, fully tested, and respects the open–closed principle for vehicles. Time-of-day logic inside DeliveryVan couples it to the clock, so tests must pass the hour explicitly (done here).
- Decision: MODIFY. Evidence: 20/20 tests pass, 0 of the 3 original tested subclasses edited, 2 small modifications.

Appendix A – Full source: ParkingApp.java (Tasks 11 and 12)

```
import java.util.*;

public class ParkingApp {

  static int hours(int minutes) { return (minutes + 59) / 60; }

  // ---------- model ----------
  interface Chargeable { double chargingFee(int kWh); }

  static abstract class Vehicle {
    private final String regNo;
    protected Vehicle(String regNo) {
      if (regNo == null || regNo.isBlank()) throw new IllegalArgumentException("regNo required");
      this.regNo = regNo;
    }
    public String getRegNo() { return regNo; }
    protected abstract double ratePerHour();
    public double calculateFee(int minutes, int entryHour) {
      if (minutes < 0) throw new IllegalArgumentException("negative duration");
      return hours(minutes) * ratePerHour();
    }
    @Override public boolean equals(Object o) { return o instanceof Vehicle v && regNo.equals(v.regNo); }
    @Override public int hashCode() { return regNo.hashCode(); }
    @Override public String toString() { return getClass().getSimpleName() + "[" + regNo + "]"; }
  }
  static class TwoWheeler extends Vehicle { TwoWheeler(String r){super(r);} protected double ratePerHour(){return 10;} }
  static class Car extends Vehicle { Car(String r){super(r);} protected double ratePerHour(){return 30;} }
  static class ElectricCar extends Car implements Chargeable {
    ElectricCar(String r){super(r);} public double chargingFee(int kWh){return kWh*12;} }
  // --- new in Task 12: only NEW classes, no existing class edited ---
  static class EBike extends Vehicle {
    EBike(String r){super(r);} protected double ratePerHour(){return 0;}
    @Override public double calculateFee(int m, int h){ if(m<0) throw new IllegalArgumentException("negative duration");
return 15; } }
  static class DeliveryVan extends Vehicle {
    DeliveryVan(String r){super(r);} protected double ratePerHour(){return 60;}
    @Override public double calculateFee(int m, int h){
      if(m<0) throw new IllegalArgumentException("negative duration");
      double rate = (h>=9 && h<17) ? 2*30 : 30;   // double the car rate, 9AM-5PM
      return hours(m)*rate; } }

  enum PassType { NONE, MONTHLY, ONE_DAY }

  static class Ticket {
    private static int counter = 1000;              // static: shared across all tickets
    final int no; final Vehicle vehicle; final int floor, slot, entryHour; final PassType pass;
    Ticket(Vehicle v, int f, int s, int h, PassType p){ no=++counter; vehicle=v; floor=f; slot=s; entryHour=h; pass=p; }
  }

  static class Facility {
    private final String[][] slots;
    private final Map<String,Ticket> active = new HashMap<>();
    private double collection = 0;
    Facility(int[] perFloor){ slots=new String[perFloor.length][]; for(int f=0;f<perFloor.length;f++) slots[f]=new
String[perFloor[f]]; }
    Ticket enter(Vehicle v, int hour, PassType p){
      if (active.containsKey(v.getRegNo())) throw new IllegalStateException("duplicate registration " + v.getRegNo());
      for(int f=0;f<slots.length;f++) for(int s=0;s<slots[f].length;s++) if(slots[f][s]==null){
        slots[f][s]=v.getRegNo(); Ticket t=new Ticket(v,f,s,hour,p); active.put(v.getRegNo(),t); return t; }
      throw new IllegalStateException("facility full");
```

```
    }
    double exit(String regNo, int minutes, int kWh){
      Ticket t = active.remove(regNo);
      if (t==null) throw new NoSuchElementException("no active ticket for " + regNo);
      slots[t.floor][t.slot]=null;
      double fee = (t.pass==PassType.NONE) ? t.vehicle.calculateFee(minutes, t.entryHour) : 0;
      if (t.vehicle instanceof Chargeable c && kWh>0) fee += c.chargingFee(kWh);
      collection += fee; return fee;
    }
    double getCollection(){ return collection; }
  }

  static Vehicle make(String type, String reg){
    return switch (type) {
      case "2W" -> new TwoWheeler(reg); case "CAR" -> new Car(reg); case "EV" -> new ElectricCar(reg);
      case "EBIKE" -> new EBike(reg); case "VAN" -> new DeliveryVan(reg);
      default -> throw new IllegalArgumentException("unknown vehicle type " + type); };
  }

  static int pass=0, fail=0;
  static void check(String name, Object expected, Object actual){
    boolean ok = expected.equals(actual); if(ok) pass++; else fail++;
    System.out.printf("%-48s expected=%-10s actual=%-10s %s%n", name, expected, actual, ok?"PASS":"FAIL"); }
  static String tryIt(Runnable r){ try { r.run(); return "no error"; } catch(RuntimeException e){ return
e.getClass().getSimpleName(); } }

  public static void main(String[] a){
    System.out.println("=== Normal operation ===");
    Facility f = new Facility(new int[]{2,1});
    Ticket t1=f.enter(make("2W","TN01A1"),10,PassType.NONE);
    Ticket t2=f.enter(make("CAR","TN02B2"),10,PassType.NONE);
    check("ticket numbers unique/increasing", true, t2.no>t1.no);
    check("2W 130 min", 30.0, f.exit("TN01A1",130,0));
    check("Car 130 min", 90.0, f.exit("TN02B2",130,0));
    f.enter(make("EV","TN03C3"),10,PassType.NONE);
    check("EV 130 min + 10 kWh", 210.0, f.exit("TN03C3",130,10));
    System.out.println("=== Boundary ===");
    Facility g = new Facility(new int[]{1});
    g.enter(make("CAR","LAST1"),10,PassType.NONE);
    check("last slot used -> next entry rejected", "IllegalStateException", tryIt(() ->
g.enter(make("CAR","X2"),10,PassType.NONE)));
    check("0 minutes", 0.0, g.exit("LAST1",0,0));
    Car c = new Car("B1");
    check("60 min (slab boundary)", 30.0, c.calculateFee(60,10));
    check("61 min (just over)", 60.0, c.calculateFee(61,10));
    check("integer-division trap 61/60*30 would give", 30.0, (61/60)*30.0);
    System.out.println("=== Invalid ===");
    Facility h = new Facility(new int[]{3});
    check("unknown type", "IllegalArgumentException", tryIt(() -> make("BUS","B1")));
    h.enter(make("CAR","DUP1"),10,PassType.NONE);
    check("duplicate registration", "IllegalStateException", tryIt(() -> h.enter(make("CAR","DUP1"),10,PassType.NONE)));
    check("exit without ticket", "NoSuchElementException", tryIt(() -> h.exit("NOPE",10,0)));
    check("negative duration", "IllegalArgumentException", tryIt(() -> new Car("N1").calculateFee(-5,10)));
    ParkingFacilityCheck.run();
    System.out.println("=== Task 12: new categories ===");
    Facility n = new Facility(new int[]{4,6,3,8,5});
    n.enter(make("EBIKE","EB1"),10,PassType.NONE);
    check("E-bike flat fee (any duration)", 15.0, n.exit("EB1",300,0));
    n.enter(make("VAN","V1"),10,PassType.NONE);
    check("Van 130 min at 10AM (double car rate)", 180.0, n.exit("V1",130,0));
    n.enter(make("VAN","V2"),20,PassType.NONE);
    check("Van 130 min at 8PM (normal car rate)", 90.0, n.exit("V2",130,0));
    n.enter(make("CAR","V3"),10,PassType.ONE_DAY);
    check("Visitor one-day pass car", 0.0, n.exit("V3",600,0));
```

```
    n.enter(make("EV","V4"),10,PassType.MONTHLY);
    check("Monthly pass EV, charging still billed", 120.0, n.exit("V4",600,10));
    System.out.println("=== Hand-calculated day scenario ===");
    Facility d = new Facility(new int[]{4,6,3,8,5});
    d.enter(make("2W","D1"),9,PassType.NONE);  d.exit("D1",45,0);      // 1h*10 = 10
    d.enter(make("CAR","D2"),9,PassType.NONE); d.exit("D2",125,0);     // 3h*30 = 90
    d.enter(make("EV","D3"),9,PassType.NONE);  d.exit("D3",60,5);      // 30+60 = 90
    d.enter(make("EBIKE","D4"),9,PassType.NONE); d.exit("D4",30,0);    // 15
    d.enter(make("VAN","D5"),10,PassType.NONE); d.exit("D5",90,0);     // 2h*60 = 120
    d.enter(make("CAR","D6"),9,PassType.MONTHLY); d.exit("D6",500,0);  // 0
    check("Day total (hand: 10+90+90+15+120+0)", 325.0, d.getCollection());
    System.out.println("TOTAL PASS=" + pass + " FAIL=" + fail);
    System.out.println("=== Benchmark: find ticket on exit (n parked) ===");
    for (int n2 : new int[]{1000, 10000, 100000}) bench(n2);
  }

  static void bench(int n){
    String[] arr = new String[n]; ArrayList<String> al = new ArrayList<>(n); HashMap<String,String> hm = new HashMap<>();
    for(int i=0;i<n;i++){ String r="TN"+i; arr[i]=r; al.add(r); hm.put(r,r); }
    String key = "TN"+(n-1); int reps = 2000; long sink=0;
    long t0=System.nanoTime(); for(int k=0;k<reps;k++){ for(int i=0;i<n;i++) if(arr[i].equals(key)){sink+=i;break;} }
    long tArr=(System.nanoTime()-t0)/reps;
    t0=System.nanoTime(); for(int k=0;k<reps;k++) sink+=al.indexOf(key); long tAl=(System.nanoTime()-t0)/reps;
    t0=System.nanoTime(); for(int k=0;k<reps;k++) sink+=hm.get(key).length(); long tHm=(System.nanoTime()-t0)/reps;
    System.out.printf("n=%-7d array scan=%-8d ns  ArrayList.indexOf=%-8d ns  HashMap.get=%-5d ns  (sink %d)%n",
n,tArr,tAl,tHm,sink>0?1:0);
  }

  static class ParkingFacilityCheck { static void run(){
    Facility x = new Facility(new int[]{1});
    check("slot freed after exit can be reused", "ok", tryIt2(x)); }
    static String tryIt2(Facility x){ x.enter(new Car("R1"),9,PassType.NONE); x.exit("R1",10,0); x.enter(new
Car("R2"),9,PassType.NONE); return "ok"; } }
}
```